



Obsidian

AUDITS

Hyperlink Security Review

Auditors

Oxjuaan

OxSpearmint

March 19, 2026

Introduction

Obsidian Audits

Obsidian audits is a team of top-ranked security researchers, with a publicly proven track record, specialising in DeFi protocols across EVM chains and Solana.

The team has achieved 10+ top-2 placements in audit competitions, placing 1st in competitions for Wormhole, Pump.fun, Yearn Finance, and many more.

Find out more: obsidianaudits.com

Audit Overview

Hyperlink is a prime broker protocol for trading on Hyperliquid.

Hyperlink engaged Obsidian Audits to perform a security review of their LST's smart contracts. The review was conducted from March 3, 2026 to March 6, 2026.

Scope

Files in scope

Repo	Files in scope
contracts Commit hash: 8e5e0bf4f852b32dc906 9981839d9f3950df830a	contracts/src/lst/*.sol contracts/src/HyperlinkTokenRelay.sol

Summary of Findings

Severity Breakdown

A total of 7 issues were identified and categorized based on severity:

- 2 Medium severity
- 3 Low severity
- 2 Informational

Findings Overview

ID	Title	Severity	Status
M-01	<code>`finalizeBatch`</code> can under-estimate the available HYPE, causing surplus HYPE to be unstaked	Medium	Fixed
M-02	Stale HyperCore delegation state in same block operations causes silent unstake failure	Medium	Fixed
L-01	<code>`HyperlinkTokenRelay`</code> lacks <code>`lastSpotBalanceChangeBlockNumber`</code> guard	Low	Fixed
L-02	<code>_requireCoreBalance</code> overestimates the available spot balance by including tokens held for limit orders	Low	Fixed
L-03	<code>`_splitWithdraws`</code> reverts on valid withdrawal amounts when the remainder is below the <code>minimumWithdrawAmount</code>	Low	Fixed
I-01	<code>StakingVault.spotSend()</code> and <code>HLHYPE.burnFrom()</code> are unreachable functions	Informational	Fixed
I-02	Unnecessary Gas Reserve Check	Informational	Fixed

Severity Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users. Alternatively, breaking a core aspect of the protocol's intended functionality
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - the attack/vulnerability requires minimal or no preconditions, but there is limited or no incentive to exploit it in practice
- **Low** - requires highly unlikely precondition states, or requires a significant attacker capital with little or no incentive.

Findings

[M-01] `finalizeBatch` can under-estimate the available HYPE, causing surplus HYPE to be unstaked

Description

`StakingVaultManager.finalizeBatch()` calculates how much HYPE is available for withdrawals using the spot balance of the `stakingVault` and `delegatorSummary.totalPendingWithdrawal`, but does not account for HYPE in the `HyperlinkTokenRelay` that has completed unstaking and is claimable via `claimUnstakedToVault`.

```
PrecompileLib.SpotBalance memory spotBalance =
stakingVault.spotBalance(stakingVault.HYPE_TOKEN_ID());

uint256 expectedSpotBalance = spotBalance.total.to18Decimals()
+ depositsInBatch;
uint256 pendingWithdrawals =
delegatorSummary.totalPendingWithdrawal.to18Decimals();
uint256 expectedAvailableForWithdraws = expectedSpotBalance +
pendingWithdrawals;
```

After the 7-day unstaking period completes, HYPE moves out of `totalPendingWithdrawal` and into `HyperlinkTokenRelay`'s spot balance, where `finalizeBatch()` does not read it. This understates `expectedAvailableForWithdraws`, causing the deficit branch to trigger with an inflated amount:

```
} else if (expectedAvailableForWithdraws < neededForWithdraws)
{
    uint256 amountToUnstake = neededForWithdraws -
expectedAvailableForWithdraws;
    stakingVault.unstake(validator,
amountToUnstake.to8Decimals());
}
```

Since `finalizeBatch()` has no access control, anyone can call it immediately after unstaking completes but before `claimUnstakedToVault` is called, front-running the keeper to force the extra unstaking. This can be repeated each cycle to perpetually keep HYPE out of delegation, reducing staking yield for all hHYPE holders.

Remediation: Fixed in PR [#33](#)

[M-02] Stale HyperCore delegation state in same block operations causes silent unstake failure

Description

HyperCore enforces a maximum of 5 pending stake withdrawals per delegator.

`HyperlinkTokenRelay.withdrawStake()` accounts for this by reading `nPendingWithdrawals` from `PrecompileLib.delegatorSummary()` and reverting if it is ≥ 5 .

However, HyperCore precompile reads return top-of-block state, any HyperCore state changes made by `CoreWriter` calls earlier in the same block are not reflected in subsequent reads within that block.

The contracts already accounts for this staleness pattern with spot balances:

`lastSpotBalanceChangeBlockNumber` enforces a one-block delay on `spotBalance()` reads. **No equivalent guard exists for delegation state.** If `nPendingWithdrawals` is 4 at the start of a block, two `unstake()` calls in that block both read 4, and both pass the ≥ 5 guard, but only the first succeeds on HyperCore, while the second silently fails because the limit was already reached.

Consider a validator with 1000 HYPE delegated, `nPendingWithdrawals = 4`, and a batch with 300 HYPE in processed withdrawals ready to finalize:

- **Tx 1 (Block N):** Owner calls `emergencyStakingWithdraw(200)`. This calls `unstake()` $\rightarrow$ `withdrawStake()`, reads stale `nPendingWithdrawals = 4`, passes the ≥ 5 check, and succeeds on HyperCore. Pending withdrawals become 5.
- **Tx 2 (Block N):** User calls `finalizeBatch()`. It updates `totalHypeProcessed += 300` and burns the escrowed hHYPE. It then calculates it must unstake 300 HYPE. It calls `unstake(300)`, and inside `withdrawStake()` the stale `nPendingWithdrawals` still reads 4, passing the ≥ 5 guard. The `CoreWriter` calls silently fail on HyperCore since the limit is already reached. The 300 HYPE is never unstaked, but all EVM state changes persist.
- **After the claim window:** Users call `claimWithdraw()`, which executes `claimAndSend()`. Since the 300 HYPE was never actually unstaked, the claim draws from the spot HYPE balance of the `HyperlinkTokenRelay` contract, funds belonging to prime broker users, not the LST.

Recommendation

Introduce a `lastDelegationChangeBlockNumber` variable in `StakingVault`, set in both `stake()` and `unstake()`. Add a `require(block.number > lastDelegationChangeBlockNumber)` guard at the top of `unstake()`, mirroring the existing `spotBalance()` staleness pattern, to prevent multiple delegation mutating operations from executing in the same block.

Remediation: Fixed in PR [#35](#)

[L-01] `HyperlinkTokenRelay` lacks `lastSpotBalanceChangeBlockNumber` guard

Description

`StakingVault` sets `lastSpotBalanceChangeBlockNumber` in every function that calls `spotSend`, and uses a `check` to prevent same-block stale reads of the `spotBalance` precompile.

`HyperlinkTokenRelay` also has several functions that also call `CoreWriterLib.spotSend` (`_disburseFromCore`, `sweepDust`, and `claimUnstakedToVault`). However, there is no check of `lastSpotBalanceChangeBlockNumber` when the `spotBalance` precompile is called. This means `_requireCoreBalance` can read stale precompile data when multiple `spotSend` calls happen in the same block.

The impact is limited since the total claimable withdrawals is not expected to exceed the relay's available tokens.

Recommendation

In the `HyperlinkTokenRelay`, consider adding a new variable `lastSpotBalanceChangeBlockNumber` and set `lastSpotBalanceChangeBlockNumber = block.number` in every code path that calls `CoreWriterLib.spotSend`. Then guard any reads from the `spotBalance` precompile to ensure that the amount returned by the precompile is up-to-date.

Remediation: Fixed in PR [#30](#)

[L-02] `_requireCoreBalance` overestimates the available spot balance by including tokens held for limit orders

Description

The `_requireCoreBalance` function checks the contract's HyperCore spot balance before a `spotSend` so that the transaction reverts if the `spotSend` would fail on HyperCore.

However, the check compares the amount being sent against `spotBalance().total`, which includes `.hold` tokens locked in open limit orders.

The actual transferable amount is `.total - .hold`, so the check overestimates available balance and could pass when the `spotSend` would fail.

Recommendation

Subtract `.hold` from `.total` to perform the check against the transferable balance:

```
function _requireCoreBalance(uint64 tokenIndex, uint64
requiredWei) private view {
-   if (PrecompileLib.spotBalance(address(this),
tokenIndex).total < requiredWei) {
+   SpotBalance memory bal =
PrecompileLib.spotBalance(address(this), tokenIndex);
+   if ((bal.total - bal.hold) < requiredWei) {
        revert Errors.InsufficientCoreSpotBalance();
    }
}
```

Remediation: Fixed in PR [#32](#)

[L-03] `\_splitWithdraws` reverts on valid withdrawal amounts when the remainder is below the minimumWithdrawAmount

Description

`queueWithdraw` splits large withdrawals into chunks of `maximumWithdrawAmount`. If the remainder after chunking falls below `minimumWithdrawAmount`, `_splitWithdraws` reverts with `WithdrawAmountNotSplittable()`.

For example, with `maximumWithdrawAmount = 10` and `minimumWithdrawAmount = 1`, a user calling `queueWithdraw(10.5 ether)` produces chunks `[10, 0.5]`. The 0.5 is below `minimumWithdrawAmount`, so the transaction reverts, even though 10.5 is a valid total withdrawal amount.

Users can work around this by manually splitting into amounts that avoid the dead zone (e.g. withdrawing 9 then 1.5).

Proof of Concept

The following test demonstrates the issue, add it to

`StakingVaultManager.t.sol`

```
    /// Demonstrates that a withdraw amount just above
    maximumWithdrawAmount
    /// reverts when the remainder falls below
    minimumWithdrawAmount.
    /// minimumWithdrawAmount = 1 HYPE (from setUp),
    maximumWithdrawAmount = 10 HYPE
    function testSplitWithdraws_RevertsWhenRemainderBelowMin()
    public {
        vm.prank(user);
        stakingVaultManager.deposit{value: 100 ether}();

        vm.prank(owner);
        stakingVaultManager.setMaximumWithdrawAmount(10
    ether);

        // 10.5 HYPE → ceil(10.5/10) = 2 chunks → [10, 0.5]
        // Last chunk 0.5 HYPE < 1 HYPE minimum → revert
        vm.startPrank(user);
        hlhype.approve(address(stakingVaultManager), 10.5
    ether);

        vm.expectRevert(ISTakingVaultManager.WithdrawAmountNotSplittable.selector);
        stakingVaultManager.queueWithdraw(10.5 ether);

        // Workaround: user splits manually into two valid
    withdrawals
        stakingVaultManager.queueWithdraw(9 ether);
        stakingVaultManager.queueWithdraw(1.5 ether);
        vm.stopPrank();
    }
```

Recommendation

Consider merging undersized remainders into the previous chunk rather than reverting.

If the current behaviour is intentional, consider documenting it clearly so integrators can avoid avoid unexpected reverts.

Remediation: Fixed in PR [#36](#)

[I-01] StakingVault.spotSend() and HLHYPE.burnFrom() are unreachable functions

Description

`StakingVault.spotSend()` and `HLHYPE.burnFrom()` are `onlyManager` functions. However, `StakingVaultManager` never calls these functions.

Additionally, the balance check always validates the vault's HYPE balance against the transfer amount, but the actual `spotSend` uses the caller supplied `token` parameter, so a non-HYPE transfer would validate against the wrong token's balance.

Recommendation

Consider removing `spotSend()` and `burnFrom()` if they are not needed.

If they are intended as admin only functions, consider changing their access control from `onlyManager` to `onlyOwner`, and fix the `spotSend` balance check to validate against the actual `token` being sent rather than `HYPE_TOKEN_ID`.

Remediation: Fixed in PR [#34](#)

[I-02] Unnecessary Gas Reserve Check

Description

`_checkGasReserve` in `_disburseFromCore` guards against HYPE gas depletion, but `withdrawWithAuthorization` and `exitWithCommitment` only perform core→core `spotSend`, which doesn't consume HYPE gas. The `gasThreshold` and `setGasThreshold` NatSpec both reference `bridgeToEvm`, but no function in `HyperlinkTokenRelay` ever calls `bridgeToEvm`, so the gas threshold check is not needed.

Recommendation

Remove `_checkGasReserve` from `_disburseFromCore` since none of the calling functions perform gas-consuming operations. If a gas reserve check is needed for future functionality, it should be applied specifically to functions that perform `bridgeToEvm` or ERC20 core→EVM transfers, not to core→core `spotSend`.

Remediation: Fixed in PR [#29](#)